

Arkitekturdokument

Grupp 13

2026-03-02

Version 0.1

Status

Granskad	Szilvia Varro-Gyapay	2026-xx-xx
Godkänd	Szilvia Varro-Gyapay	2026-xx-xx

Projektidentitet

Grupp E-post: tddd96_2026vt_ax_13@groups.liu.se

Beställare: Johan Munck, Börjes Koncernen
Tfn: +46733318552
E-post: johan@borjeskoncernen.se

Handledare: Szilvia Varro-Gyapay
Tfn: +4613284092
E-post: szilvia.varro-gyapay@liu.se

Kursansvarig: Lena Buffoni
Tfn: +4613284046
E-post: lena.buffoni@liu.se

Projektdeltagare

Namn	Ansvar	E-post
Karl Lennartsson	Teamledare	karle221@student.liu.se
Simon Andersson	Dokumentansvarig	siman020@student.liu.se
Elias Facq	Analysansvarig	eliwa076@student.liu.se
Jacob Ahling Hult	Utvecklingsledare	jacah067@student.liu.se
Jakob Renqvist	Arkitekt	jakre732@student.liu.se
Viktor Wallberg	Kvalitetssamordnare	vikwa448@student.liu.se
Felix Hallström	Testledare	felha291@student.liu.se

INNEHÅLL

1	Inledning	1
2	Systemarkitektur	1
2.1	Vår DB	1
2.2	Backend	3
2.3	Frontend	3
2.4	API och iLog	4
3	Funktionella krav	4
4	Systemanatomi	4

DOKUMENTHISTORIK

Version	Datum	Utförda förändringar	Utförda av	Granskad
0.1	2026-02-16	Första utkast	Jakob Renqvist	2026-02-18
0.2		Första revision	Grupp 13	
1.0		Första version	Grupp 13	

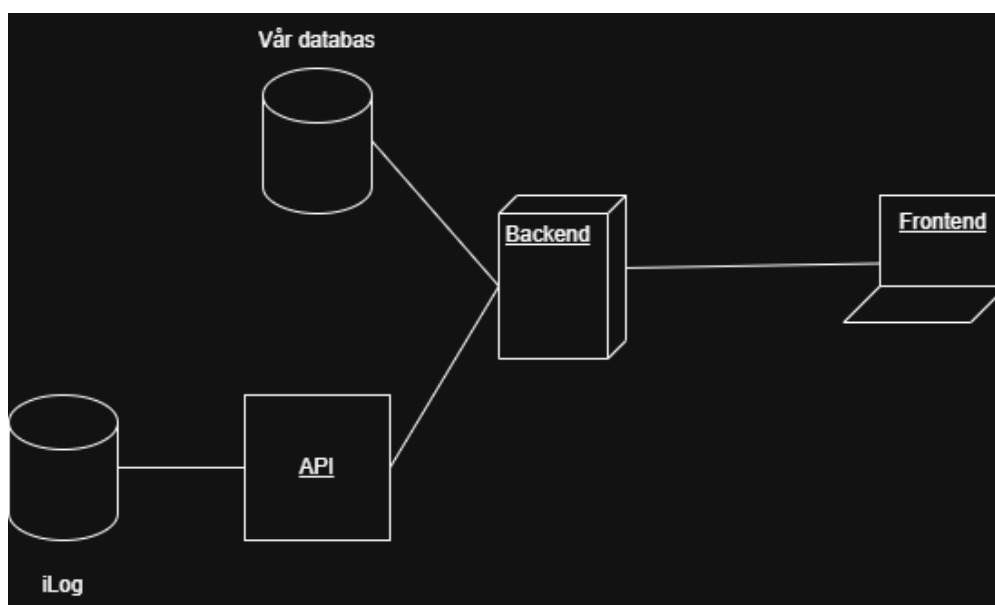
1 INLEDNING

Denna arkitektbeskrivning beskriver en överliggande arkitektur för att utveckla ett system för Börjes Koncernen. Systemet skall underlätta för trafikledare att göra avgörande beslut inom logistiken hos kundföretag. Detta genom att ta hänsyn till historisk data och kostnader för frakter för att göra en estimering för kostnaden hos en framtida frakt.

2 SYSTEMARKITEKTUR

I detta kapitel diskuteras den generella arkitekturen över systemet detta projekt skall utveckla. I figur 1 visas ett diagram över systemarkitekturen. De system detta projekt kommer fokusera på är *Vår databas*, *Backend* samt *Frontend* enligt figur. De resterande komponenterna (*API* och *iLog*) skall redan vara utvecklade och färdigställda av kunden. I följande underkapitel diskuteras samtliga komponenter i detalj.

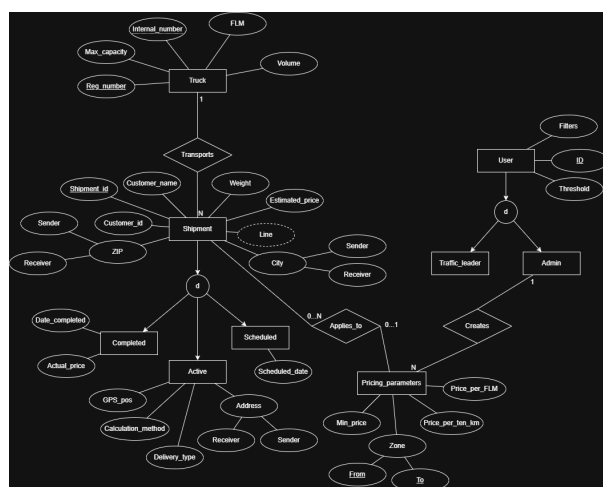
kanske förklara trappstegsmodell och skit



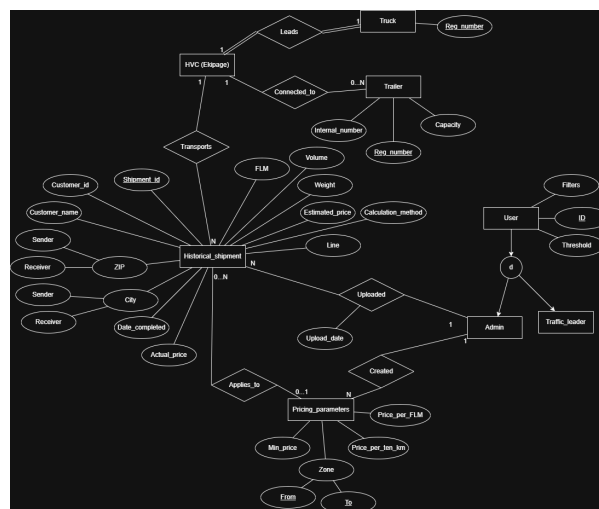
Figur 1: Systemarkitektur över alla komponenter. Detta är version 1 utgivet 5 feb 2026.

2.1 Vår DB

Den data databasen detta projekt kommer utveckla har justerats och förändrats en hel del fram till dagens datum (2026-02-26). Därför presenteras två versioner av ett *Enhanced Entity-Relationship model* (kallat EER-diagram i denna rapport) i figur 2a och figur 2b. Ett EER-diagram är ett välkänt och standardiserat arbetsverktyg för att utveckla databaser. Att sedan översätta detta diagram till en faktisk relationsdatabas är också standardiserat. Men eftersom detta diagram fortfarande undergår stora förändringar så kommer denna översättning att ske när diagrammet är mognare. Notationen och vad allt betyder i ett EER-diagram kommer ej förklaras i denna rapport (hade varit ganska lång).



(a) Version 1



(b) Version 3

Figure 2: EER-diagram över databasen som ska utvecklas.

I figur 2a visas ett första utkast på EER-diagrammet. I detta diagram är det tänkt att vi ska spara lastbilar (*Truck* i diagram) med attributer som registreringskylt osv. En (1) lastbil ska kunna transportera flera (N) frakter (*Shipment* i diagram) vilket ger en 1:N relation. En frakt ska också specialiseras disjunkt till klar (*Completed*), aktiv (*Active*) och schemalagd (*Scheduled*). Vidare ska databasen också spara användare som också specialiseras disjunkt till trafikledare (*Traffic_leader*) och administratör (*Admin*). Det en trafikledare ska kunna göra i systemet har inget med databasen att göra, så den enheten har inga attributer eller relationer. En administratör ska kunna skapa en generell prisparameter som sista steg i trappstegsmodellen som i sin tur kan eller kan inte (ty 0...1 och 0..N restriktionerna) ha en relation med en frakt.

I figur 2b visas version 3 av EER-diagrammet. Två stora förändringar har skett i samband med ett flertal mindre förändringar. En av de stora förändringarna är att enheten *Truck* har delats upp mer specifikt så vi har lastbildshuvudet som kan kopplas till ett ekipage samt N antal släp (*Trailer*) som också kan kopplas till samma ekipage. Ett ekipage har sedan en relation med en historisk frakt (*Historical_shipment*) som är del av den andra stora förändringen. I version 1 var en frakt delad i tre möjliga specialiseringar. Men detta kan vara onödigt då system som hanterar schemalagda frakter och aktiva frakter redan hanteras i kundens existerande system. Dessa två specialiseringar kan då tas bort vilket hade gjort att en frakt endast kan vara historiskt. Därav har namngivningen för en frakt (*Shipment* i v1) ändrats till *Historical_shipment* i v3. Attributerna för *Completed*-frakter har lagts till i *Historical_shipment* medan attributerna för de andra specialiserade enheterna har tagits bort.

De mindre förändringarna kommer ej diskuteras i större omfattning, men här är en log på alla förändringar mellan de tre nuvarande versionerna:

Version 1 (2026-02-05)

Första utgåva. Inga möjliga förändringar från tidigare.

Version 2 (2026-02-09)

- Flyttade attributen `FLM` och `Volume` från `Truck` till `Shipment`.
- Döpte om attributen `Max_capacity` i `Truck` till `Capacity`.
- Flyttade attributen `Calculation_method` från `Active` till `Shipment`.
- Ändrade attributen `Line` till att inte vara `deriverat`.
- Lade till relationen `Uploaded` mellan `Admin` och `Completed`.

Version 3 (2026-02-12)

- Omstrukturerade `Truck` till tre separata entiteter.
- Tog bort specialiseringen av `Shipment` till `Active` och `Scheduled`.
- Döpte om `Shipment` till `Historical_shipment`.
- Lade till attributen `Date_completed` och `Actual_price` i `Historical_shipment`.
- Kopplade om relationen `Uploaded` till `Historical_shipment`.

2.2 Backend

Per dagens datum har inte mycket arkitekturellt utvecklats på backenden av projektet. Val av programmeringsspråk är fortfarande under diskussion i gruppen. Kunden har gett kravet att kommunikation mellan backend och frontend skall ske säkert med kryptering, HTTPS osv. Programmatiskt om backend ska använda krypteringsmetoder som JSON Web Token (JWT), OAuth eller möjligtvis en ekvivalent till Cloudflare har ej diskuterats ännu. Diskussion om detta med kunden skall ske i framtiden.

2.3 Frontend

Tidiga utkast på designen för frontend hade gjorts innan ett första kundmöte hade genomförts. Dessa utkast har i princip skrotats och endast abstrakta förklaringar (inga diagram/ritningar/bilder) finns för arkitekturen.

En användare ska kunna logga in via en inloggningsvy likt den som finns på `ilog.btf.se`. En lista över systemets *functional requirements* har skapats som till större del handlar om systemets frontend-krav.

Detaljer om exakt hur systemets frontend skall se ut, som möjliga vyer, design, utseende osv, har ej utvecklats ännu. Många punkter i listan över funktionella krav i kapitel 3 beskriver hur frontend skall fungera. Översiktligt så skall en trafikledare kunna logga in i vårt system via en inloggningsvy. Efter inloggning ska trafikledaren kunna se en lista över de frakter hen har ansvar över. Denna lista ska också kunna filtreras efter vissa kriterier. Möjligtvis också att en sökningsfunktion ska göra det enklare för trafikledaren att hitta rätt frakt. Om en frakt i denna lista klickas så expanderar vyn och mer detaljer om frakten visas, bland annat vår prisestimering. En trafikledare skall också kunna simulera olika fall genom att lägga till, ta bort, kombinera, fixa med olika frakter för att se vad som kan maximera lönsamheten för vår prisestimering. Denna simulering ska kunna ske för en existerande frakt eller så kan en helt tom frakt användas för simulering.

2.4 API och iLog

API:et och iLog som finns i figur 1 är system som detta projekt ej ska utveckla. Dessa system ska presenteras av kunden. Vid dagens datum är detta API:et fortfarande under utveckling, så dokumentation/apinyklar osv har vi ej tillgång till.

iLog är en stor databas med kundens alla frakter. iLog är den databas trafikledare rapporterar in och ser schemalagda frakter genom befintliga system. Systemet detta projekt utvecklar ska genom API:et kunna hämta relevant data från iLog.

3 FUNKTIONELLA KRAV

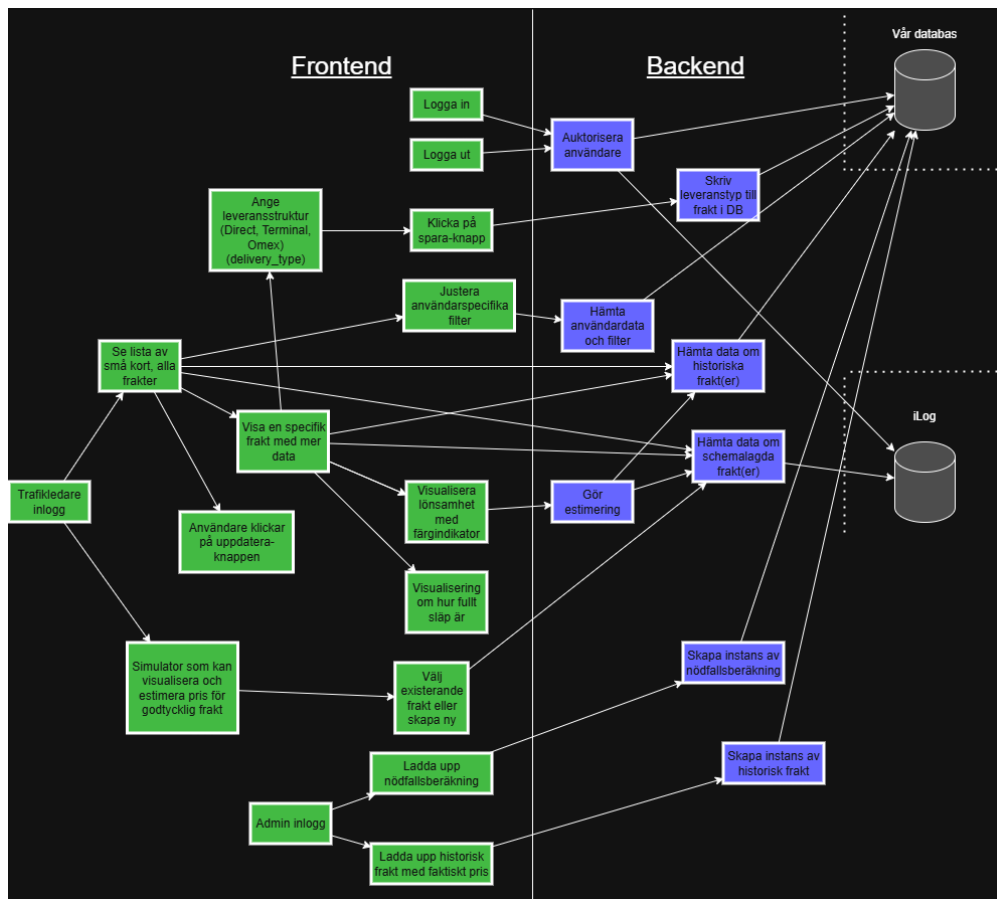
I detta kapitel beskrivs *functional requirements* för systemet vilket ger en överblick över hur hela systemet skall fungera

1. Bil + Släp ska presenteras som ett kort. Ska hämta aktiv och schemalagd data från iLog.
2. Visualisera hur fullt ett ekipage är. Om bokad flakmeter > kapacitet ska detta också visualiseras
3. Ska kunna se lönsamhetssiffra/intäkt med färgindikator baserat på tröskelvärde.
4. När ett kort klickas så ska mer info om den schemalagda frakten (data från iLog) samt vårt estimerade pris m.m visas.
5. Trafikledare ska kunna ange leveransstruktur (Direct, Terminal, Omex osv) för schemalagd frakt.
6. En simulator där trafikledare kan mata in fiktiva bokningar eller ändra på existerande bokningar (temporärt och ej faktiska ändringar som sparas) för att se effekt.
7. Trafikledare ska kunna klicka på en uppdateraknapp som hämtar schemalagda frakter i iLog som presenteras som kort i frontend.
8. Prisestimeringen ska kunna ske genom en trappstegsmodell med (minst) tre olika sätt att räkna ut priset.
9. Systemet ska designas för hög sökerhet och minimal administration.
10. Systemet ska bara läsa data från iLog, inte skriva.
11. Trafikledare har en speciell vy där filter som regioner, tröskelvärde m.m kan redigeras.
12. Admin ska kunna ladda upp fil med historiska sändningar inklusive faktiska priset för den sändningen.
13. Admin ska kunna skapa åorst casefall i sista steg i trappstegsmodell.

4 SYSTEMANATOMI

I figur 3 visas ett diagram över hur systemet i sin helhet ska se ut. De gröna fyrhörningarna är processer som ska ske på frontend-sidan av systemet medan blåa fyrhörningar är processer som ska ske på backend-sidan av systemet.

Mest noterbart finns det fyra rotnoder på frontend-sidan: Logga in", Logga ut", Trafikledare inlogg och Admin inlogg". Resterande noder visar under underprocesser till dessa fyra rotnoder som alla resulterar i anrop till antingen vår databas eller iLog-databasen. Funktionella kraven i kapitel 3 har använts som utgångspunkt vid skapandet av detta diagram.



Figur 3: Systemanatomy